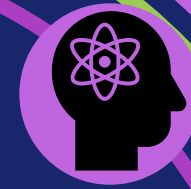
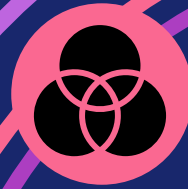


2026年度 前期 データマイニング特論 (I20教室)  
第9回 2026年6月11日(木) 13:10~14:40



# データマイニング特論

第9回

クラスタリング法

## 本科目シラバスの確認 3（授業計画）

| #    | 日程      | 内容                         |
|------|---------|----------------------------|
| 第1回  | 4/16（木） | ガイダンス、イントロダクション+Python環境構築 |
| 第2回  | 4/23（木） | pandasによるデータ操作の基礎          |
| 第3回  | 4/30（木） | 探索的データ分析（EDA）と可視化（オンデマンド）  |
| 第4回  | 5/7（木）  | データの前処理と特徴量エンジニアリング        |
| 第5回  | 5/14（木） | 分類の基礎 — k近傍法と決定木           |
| 第6回  | 5/21（木） | アンサンブル学習                   |
| 第7回  | 5/28（木） | 線形分類モデルと回帰                 |
| 第8回  | 6/4（木）  | モデル評価・選択と不均衡データ            |
| 第9回  | 6/11（木） | クラスタリング手法                  |
| 第10回 | 6/18（木） | アソシエーション分析と異常検知            |
| 第11回 | 6/25（木） | テキストマイニング                  |
| 第12回 | 7/2（木）  | 時系列データマイニング                |
| 第13回 | 7/9（木）  | 深層学習の概観と使いどころ              |
| 第14回 | 7/16（木） | 公平性・倫理と再現性                 |
| 第15回 | 7/23（木） | まとめ                        |

【Point】 やむなく欠席した場合、資料を確認のうえ不明ところは聞いてください。

# 1



## クラスタリング

教師なし学習でデータの構造を見つける

# 『教師あり学習』から『教師なし学習』へ

【PR-AUCやF1スコアは、もう使えない。。。】

## 教師あり学習

- ・ 正解ラベルがある（例：SHか非SHか）
- ・ ラベルを予測する
- ・ 精度 / PR-AUC / F1で評価できる
- ・ 「どれだけ正解に近いか」を測る

## 教師なし学習

- ・ 正解ラベルがない
- ・ 似たものの同士をまとめる
- ・ 精度では評価できない
- ・ グループの意味は人間が解釈する



# クラスタリングとは

【正解ラベルのないデータから「似たもの同士のグループ（クラスター）」を見つける】

## 顧客セグメンテーション

- ・似た購買行動の顧客をグループ化してマーケティングに活用

## 異常検知

- ・どのグループにも属さないものを異常とみなす

## データの要約

- ・大量のデータを少数の代表パターンに圧縮する

「似ている」をどう測るか → 距離

⇒ユークリッド距離（直線距離）が最も基本的。

⇒距離が近い点ほど「似ている」とみなす。

研究での応用例：遺伝子発現パターンで似た遺伝子をグループ化、  
群集組成でサンプルを分類

# 本日の環境準備 (I)

いろいろ試してみよう！

# 日本語対応のモジュールをインストールする(その1)

```
!pip install japanize-matplotlib
```

# 日本語対応のモジュールをインストールする(その2)

```
!apt-get install -y fonts-ipafont-gothic
```

# 各ライブラリーをインポートする

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import japanize_matplotlib
```

```
import warnings
```

```
warnings.filterwarnings("ignore")
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.cluster import KMeans, AgglomerativeClustering, DBSCAN
```

```
from sklearn.metrics import silhouette_score, adjusted_rand_score
```

```
from scipy.cluster.hierarchy import dendrogram, linkage, fcluster
```

## 本日の環境準備（２）

いろいろ試してみよう！

## # データ読み込み

```
URL = "https://data.insideairbnb.com/japan/kant%C5%8D/tokyo/2023-06-29/data/listings.csv.gz"
df = pd.read_csv(URL, compression="gzip", low_memory=False)
```

## # 価格の数値化

```
if df["price"].dtype == object:
    df["price"] = df["price"].str.replace(r"[$,]", "", regex=True).astype(float)
```

## # スーパーホストラベル(最後の発展課題でのみ使う。クラスタリングには使わない)

```
df["is_superhost"] = (df["host_is_superhost"] == "t").astype(int)
```

```
FEATURES = [
    "price", "number_of_reviews", "review_scores_rating",
    "accommodates", "bedrooms", "minimum_nights", "availability_365",
]
```

```
df_clean = df[FEATURES + ["is_superhost"]].dropna()
```

## # 計算を軽くするためサンプリング(任意)

```
if len(df_clean) > 3000:
    df_clean = df_clean.sample(3000, random_state=42).reset_index(drop=True)
```

```
X = df_clean[FEATURES].values
```

```
y_sh = df_clean["is_superhost"].values # 発展課題用(クラスタリングには使わない)
```

```
print(f"データ準備完了:{len(df_clean):,} 件、{len(FEATURES)} 変数")
```

```
print("※ クラスタリングは教師なし学習なので is_superhost は使わない")
```

```
print("（最後の発展課題でのみ照合に使う）\n")
```

データ準備完了：3,000 件、7 変数

※ クラスタリングは教師なし学習なので is\_superhost は使わない  
(最後の発展課題でのみ照合に使う)

## 本日の環境準備（3）

いろいろ試してみよう！

# 標準化前の各変数のスケールを確認

```
print("— 標準化前の各変数のスケール —")
```

```
scale_df = pd.DataFrame({  
    "平均": df_clean[FEATURES].mean(),  
    "標準偏差": df_clean[FEATURES].std(),  
    "最大値": df_clean[FEATURES].max(),  
})
```

```
print(scale_df.round(1))
```

```
scaler = StandardScaler()
```

```
X_sc = scaler.fit_transform(X)
```

```
print("¥n→ price(数千～数万)と review_scores_rating(0～5)では")
```

```
print(" スケールが全く違う。標準化しないと price だけで距離が決まる")
```

```
print("→ StandardScalerで平均0・分散1に揃える¥n")
```

— 標準化前の各変数のスケール —

|                      | 平均      | 標準偏差    | 最大値       |
|----------------------|---------|---------|-----------|
| price                | 21897.2 | 37950.2 | 1052387.0 |
| number_of_reviews    | 41.9    | 58.0    | 799.0     |
| review_scores_rating | 4.7     | 0.4     | 5.0       |
| accommodates         | 4.5     | 2.9     | 16.0      |
| bedrooms             | 1.5     | 1.0     | 11.0      |
| minimum_nights       | 2.1     | 6.4     | 180.0     |
| availability_365     | 141.4   | 101.8   | 365.0     |

→ price (数千～数万) と review\_scores\_rating (0～5) では  
スケールが全く違う。標準化しないと price だけで距離が決まる

→ StandardScalerで平均0・分散1に揃える



The background is a gradient of dark blue and purple, speckled with small white dots. On the left side, there is a large, semi-circular scale with tick marks and numbers ranging from 140 to 260. Several concentric circles and arcs are scattered across the image, some with arrows indicating a clockwise direction. The text 'k-means' is centered in the middle-right area.

**k-means**

# k-means (1)

## 【k-meansの仕組み — 重心ベースの反復】

Step 1 : 初期化

⇒ k個の重心をランダムに配置する

Step 2 : 割り当て

⇒ 各点を最も近い重心のクラスに割り当てる

Step 3 : 重心更新

⇒ 各クラスに属する点の平均位置に重心を移動する

Step 4 : 収束

⇒ 重心が動かなくなるまでStep 2-3を繰り返す

## k-meansの3つの前提・限界

① クラスタ数kを事前に決める

⇒ いくつに分けるかを人間が指定する必要がある

② 球状のクラスを仮定

⇒ 細長い・三日月形などのクラスは苦手

③ 外れ値に弱い

⇒ 極端な点が重心を引っ張ってしまう

## k-means (2)

いろいろ試してみよう！

```
# k-means
km = KMeans(n_clusters=3, random_state=42, n_init=10)
labels_km = km.fit_predict(X_sc)

df_clean["cluster_km"] = labels_km

print("— 各クラスタのサイズ —")
print(pd.Series(labels_km).value_counts().sort_index())
print(f"¥nInertia(クラスタ内誤差平方和): {km.inertia_:.1f}")

# 各クラスタの特徴(元のスケールで平均を確認)
print("¥n— 各クラスタの特徴(変数ごとの平均)—")
cluster_summary = df_clean.groupby("cluster_km")[FEATURES].mean()
print(cluster_summary.round(1).to_string())
```

— 各クラスタのサイズ —

0 2486

1 507

2 7

Name: count, dtype: int64

Inertia (クラスタ内誤差平方和) : 13555.0

— 各クラスタの特徴 (変数ごとの平均) —

|            | price    | number_of_reviews | review_scores_rating | accommodates | bedrooms | minimum_nights | availability_365 |
|------------|----------|-------------------|----------------------|--------------|----------|----------------|------------------|
| cluster_km |          |                   |                      |              |          |                |                  |
| 0          | 16812.3  | 36.1              | 4.6                  | 3.5          | 1.1      | 1.9            | 142.7            |
| 1          | 38435.6  | 70.9              | 4.7                  | 9.5          | 3.1      | 2.0            | 134.7            |
| 2          | 629892.1 | 20.6              | 4.8                  | 3.7          | 1.0      | 103.3          | 174.3            |

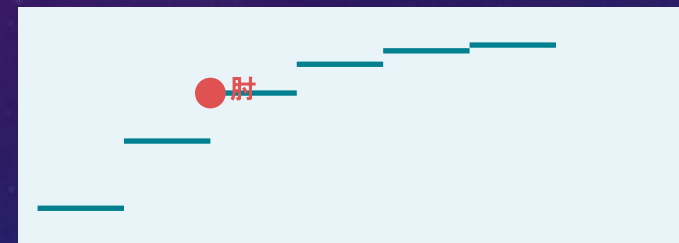


# クラスタ数決定 (1)

【k-means最適なクラスタ数kの決め方】

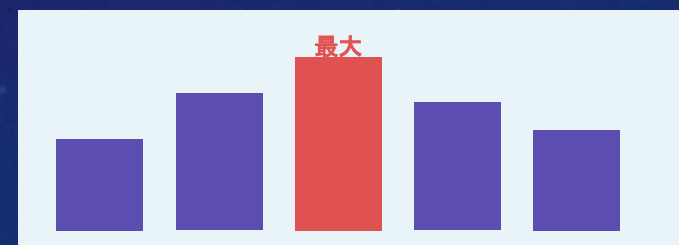
## エルボー法

- ⇒ kを増やしながらinertia (クラスタ内誤差平方和) を計算する。
- ⇒ 減少が緩やかになる「肘 (elbow)」の位置を最適なkとする。



## シルエット分析

- ⇒ 各点が「自分のクラスタへの適合」と「隣のクラスタからの距離」をスコア化する。
- ⇒ シルエット係数が最大になるkを選ぶ。
  - 1~1の値をとる。



どちらも万能ではない。

最終的にはドメイン知識 (解釈できるか) と組み合わせて判断する。



## クラスタ数決定 (2)

いろいろ試してみよう！

# 最適なクラスタ数kの決定

inertias, silhouettes = [], []

k\_range = range(2, 11)

for k in k\_range:

km\_k = KMeans(n\_clusters=k, random\_state=42, n\_init=10)

lab = km\_k.fit\_predict(X\_sc)

inertias.append(km\_k.inertia\_)

silhouettes.append(silhouette\_score(X\_sc, lab))

#次に続く。。。

## クラスタ数の決定 (3)

いろいろ試してみよう！

## # グラフ

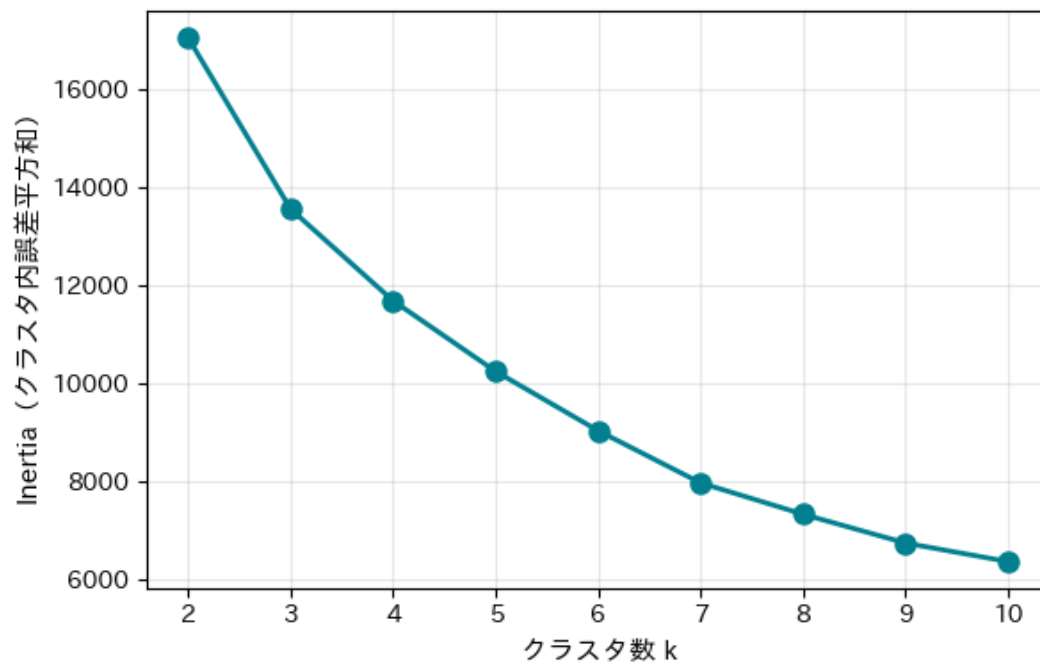
```
fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))
axes[0].plot(list(k_range), inertias, "o-", color="#028090", linewidth=2, markersize=8)
axes[0].set_xlabel("クラスタ数 k", fontsize=11)
axes[0].set_ylabel("Inertia(クラスタ内誤差平方和)", fontsize=11)
axes[0].set_title("エルボー法(肘の位置が最適なk)", fontsize=12)
axes[0].grid(True, alpha=0.3)
axes[1].plot(list(k_range), silhouettes, "o-", color="#5C4DB1", linewidth=2, markersize=8)
best_k = list(k_range)[int(np.argmax(silhouettes))]
axes[1].axvline(x=best_k, color="#E05252", linestyle="--", label=f"最大: k={best_k}")
axes[1].set_xlabel("クラスタ数 k", fontsize=11)
axes[1].set_ylabel("シルエット係数", fontsize=11)
axes[1].set_title("シルエット分析(最大が最適なk)", fontsize=12)
axes[1].legend(fontsize=10)
axes[1].grid(True, alpha=0.3)
plt.suptitle("最適なクラスタ数kの決定", fontsize=13)
plt.tight_layout()
plt.savefig("choose_k.png", dpi=150, bbox_inches="tight")
plt.show()

print(f"¥nシルエット係数が最大のk: {best_k}")
print("各kのシルエット係数:")
for k, sil in zip(k_range, silhouettes):
    mark = " ← 最大" if k == best_k else ""
    print(f"  k={k}: {sil:.3f}{mark}")
```

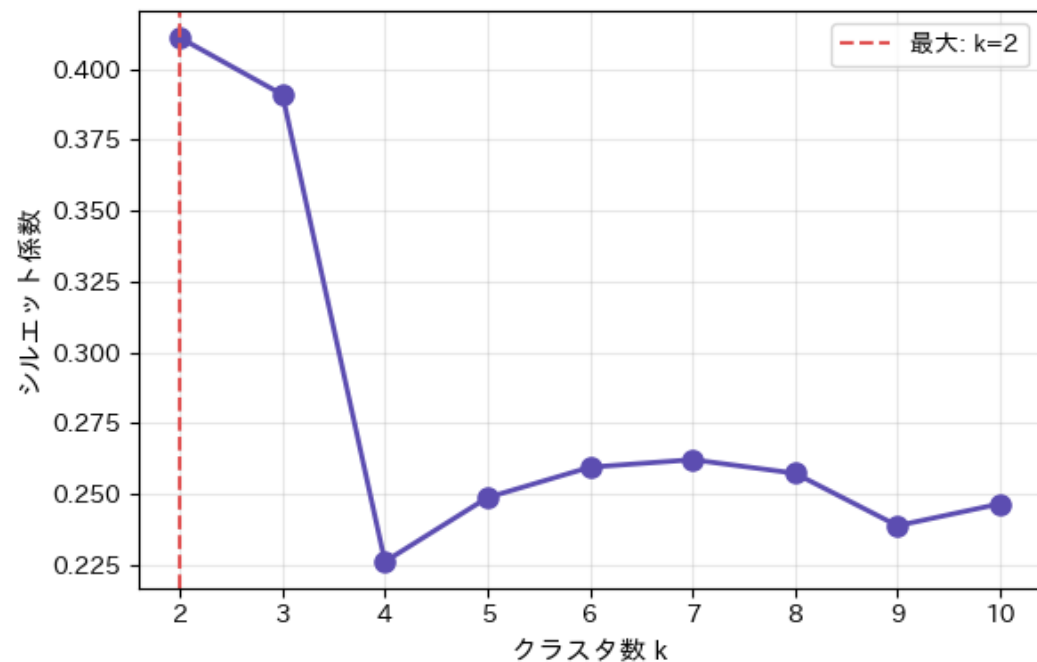
## クラスタ数決定 (4)

最適なクラスタ数kの決定

エルボー法 (肘の位置が最適なk)



シルエット分析 (最大が最適なk)



シルエット係数が最大のk: 2

各kのシルエット係数:

k=2: 0.411 ← 最大

k=3: 0.391

k=4: 0.226

k=5: 0.249

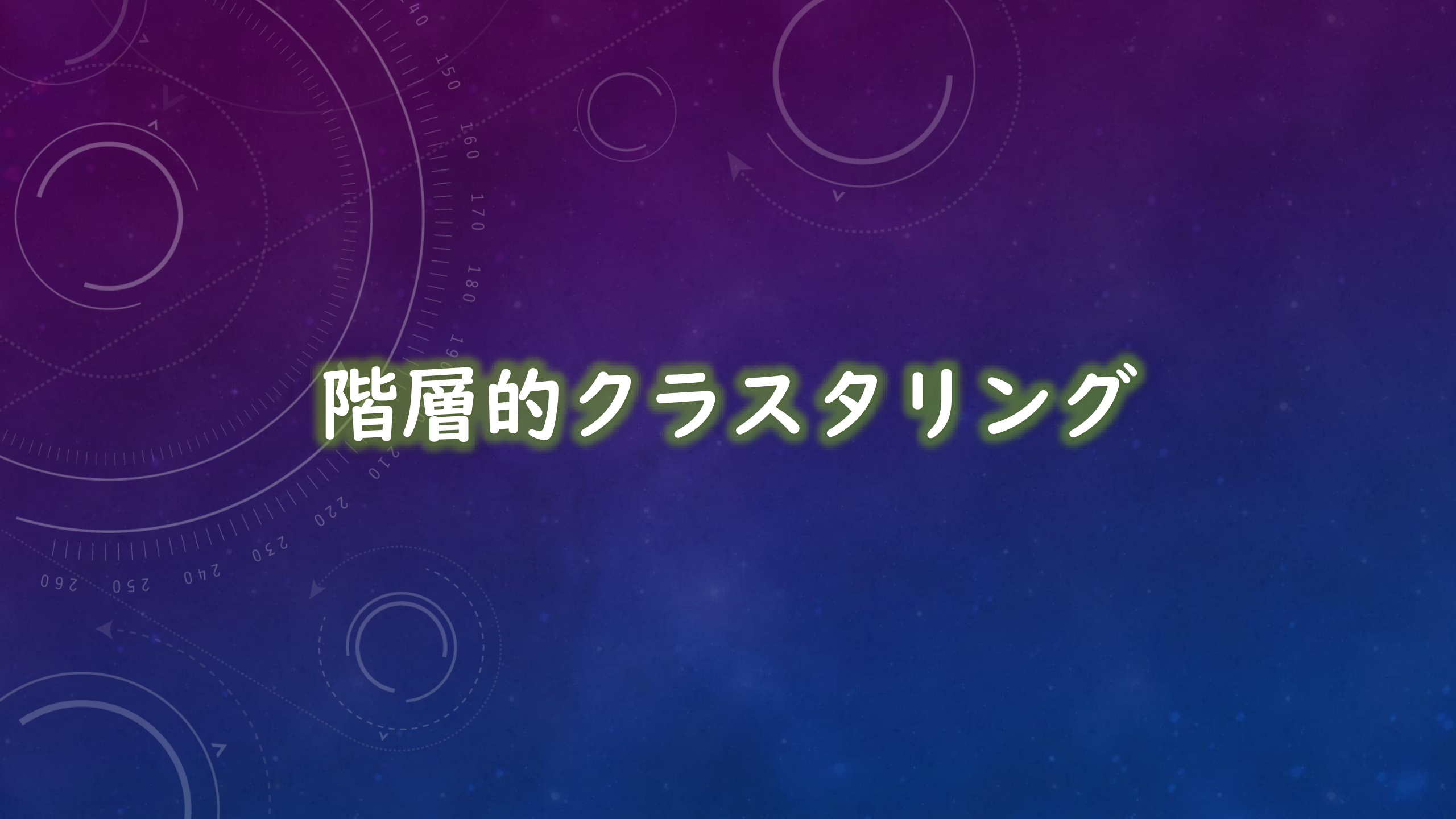
k=6: 0.260

k=7: 0.262

k=8: 0.257

k=9: 0.239

k=10: 0.247

The background features a dark blue gradient with a subtle pattern of white stars. Overlaid on this are several faint, light blue circular elements. These include concentric circles, arcs, and dashed lines, some of which have small arrows indicating a clockwise direction. A prominent circular scale is visible on the left side, with numerical markings ranging from 140 to 260 in increments of 10. The overall aesthetic is technical and futuristic.

# 階層的クラスタリング



# 階層的クラスタリング（Ⅰ）

## 【階層的クラスタリング — 樹形図】

### デンドログラム（樹形図）

⇒最も近い点同士を順番に併合

⇒任意の高さで切ってクラスター数を決める

### 連結方法

ward

：クラスタ内分散を最小化。（最もよく使われる）

⇒バランスの良いクラスタができる

complete

：クラスタ間の最も遠い点同士の距離で併合。

⇒コンパクトなクラスタ

average

：クラスタ間の全点ペアの平均距離で併合。

⇒中間的な性質

## 階層的クラスタリング (2)

いろいろ試してみよう！

# デンドログラムと階層的クラスタリング

# ward法で連結

Z = linkage(X\_sc, method="ward")

# デンドログラムの描画

plt.figure(figsize=(12, 5))

dendrogram(Z, truncate\_mode="lastp", p=30, leaf\_rotation=90, leaf\_font\_size=9)

plt.axhline(y=40, color="#E05252", linestyle="--", linewidth=1.5, label="切断位置(例)")

plt.xlabel("サンプル(下位30クラスタに集約)", fontsize=11)

plt.ylabel("距離(ward)", fontsize=11)

plt.title("デンドログラム(ward法)", fontsize=12)

plt.legend(fontsize=10)

plt.tight\_layout()

plt.savefig("dendrogram.png", dpi=150, bbox\_inches="tight")

plt.show()

# 3クラスタに分割

agg = AgglomerativeClustering(n\_clusters=3, linkage="ward")

labels\_hc = agg.fit\_predict(X\_sc)

df\_clean["cluster\_hc"] = labels\_hc

print("¥n— 階層的クラスタリング(3クラスタ)のサイズ —")

print(pd.Series(labels\_hc).value\_counts().sort\_index())

# k-meansとの比較

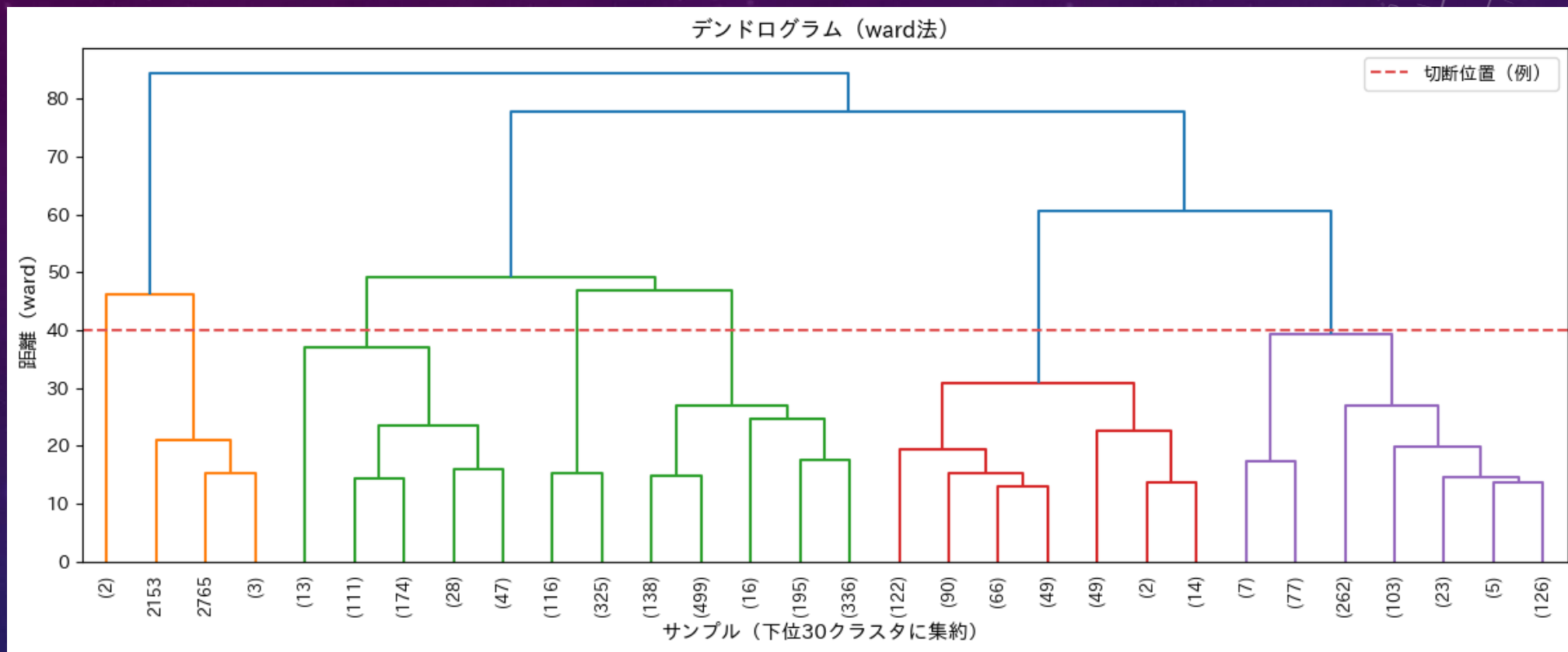
print("¥n— k-means と階層的クラスタリングの一致度 —")

ari = adjusted\_rand\_score(labels\_km, labels\_hc)

print(f"調整ランド指数(ARI): {ari:.3f}")

print(" (1に近いほど2つのクラスタリング結果が一致)")

## 階層的クラスタリング (3)



ARI は Adjusted Rand Index (調整ランド指数) の略で、  
2つのクラスタリング結果がどれくらい一致しているかを測る指標

—— 階層的クラスタリング (3クラス) のサイズ ——

0 995

1 7

2 1998

Name: count, dtype: int64

—— k-means と階層的クラスタリングの一致度 ——

調整ランド指数 (ARI) : 0.408

(1に近いほど2つのクラスタリング結果が一致)

The background is a gradient of dark blue and purple, speckled with small white dots. On the left side, there are several concentric circles and a large circular scale with degree markings from 140 to 260. Some of the circles have arrows indicating a clockwise direction. The text 'DBSCAN' is centered in the middle of the image.

**DBSCAN**



# DBSCAN (I)

## 【DBSCAN — 密度ベースのクラスタリング】

任意の形状を検出できる

⇒ k-meansは球状しか見つけられないが、  
DBSCANは三日月形・リング状も検出する。

外れ値を自動検出する

⇒ どのクラスタにも属さない点を「ノイズ」として分離。  
⇒ 異常検知に直結する。

2つのパラメータ（クラスタ数は指定しない — k-meansとの大きな違い）

eps（イプシロン）：近傍とみなす半径。

この距離内に点が密集していれば同じクラスタ。

⇒ 大きすぎると全部1つ、小さすぎると全部ノイズに

min\_samples

：クラスタの核（コア点）とみなす最小の近傍点数。

⇒ 大きいほど厳しい基準でクラスタを作る

## DBSCAN (2)

いろいろ試してみよう！

# DBSCAN(クラスタ数を指定しない)

```
db = DBSCAN(eps=0.5, min_samples=5)
labels_db = db.fit_predict(X_sc)
df_clean["cluster_db"] = labels_db
```

# ラベル -1 はノイズ(外れ値)

```
n_clusters = len(set(labels_db)) - (1 if -1 in labels_db else 0)
n_noise = list(labels_db).count(-1)
```

```
print(f"— DBSCAN(eps=0.5, min_samples=5)—")
print(f"検出されたクラスタ数: {n_clusters}")
print(f"ノイズ(外れ値)の数: {n_noise} ({n_noise/len(labels_db)*100:.1f}%)")
```

# epsを変えて結果を比較

```
print("\n— epsを変えたときの結果 —")
for eps in [0.3, 0.5, 0.8, 1.2, 1.5]:
    db_e = DBSCAN(eps=eps, min_samples=5)
    lab_e = db_e.fit_predict(X_sc)
    nc = len(set(lab_e)) - (1 if -1 in lab_e else 0)
    nn = list(lab_e).count(-1)
    print(f"  eps={eps}: クラスタ{nc}個, ノイズ{nn}件 "
          f"({nn/len(lab_e)*100:.1f}%)")
```

```
— DBSCAN (eps=0.5, min_samples=5) —
検出されたクラスタ数: 17
ノイズ(外れ値)の数: 981 (32.7%)
```

```
— epsを変えたときの結果 —
```

```
eps=0.3: クラスタ38個, ノイズ2090件 (69.7%)
eps=0.5: クラスタ17個, ノイズ981件 (32.7%)
eps=0.8: クラスタ7個, ノイズ406件 (13.5%)
eps=1.2: クラスタ4個, ノイズ144件 (4.8%)
eps=1.5: クラスタ3個, ノイズ61件 (2.0%)
```

# 2

## 3手法の比較

---

どれを選ぶか





# 3手法の比較（1）

## 【3手法の比較 — どれを選ぶか】

「どれが最良か」ではなく「データの性質と目的に応じて選ぶ」。  
球状で高速ならk-means、外れ値検出ならDBSCAN。

| 観点      | k-means | 階層的    | DBSCAN   |
|---------|---------|--------|----------|
| クラスター数  | 事前に指定   | 後から決定  | 自動（指定不要） |
| クラスター形状 | 球状のみ    | 柔軟     | 任意の形状    |
| 外れ値     | 弱い      | 弱い     | 自動検出     |
| 計算速度    | 速い      | 遅い     | 中間       |
| 大規模データ  | ◎       | △      | ○        |
| 解釈のしやすさ | 中間      | 樹形図で明快 | 中間       |



# 3手法の比較（2）

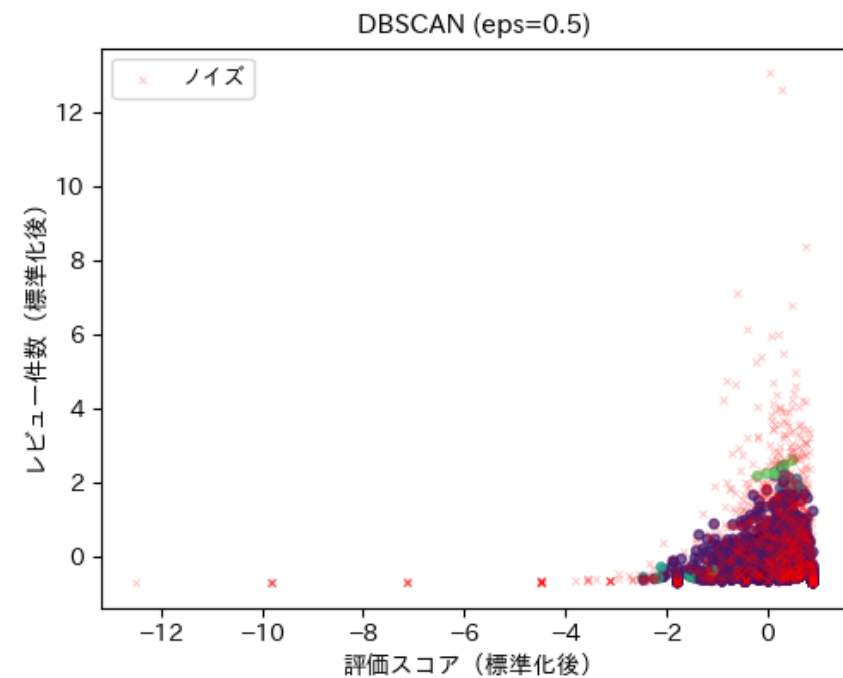
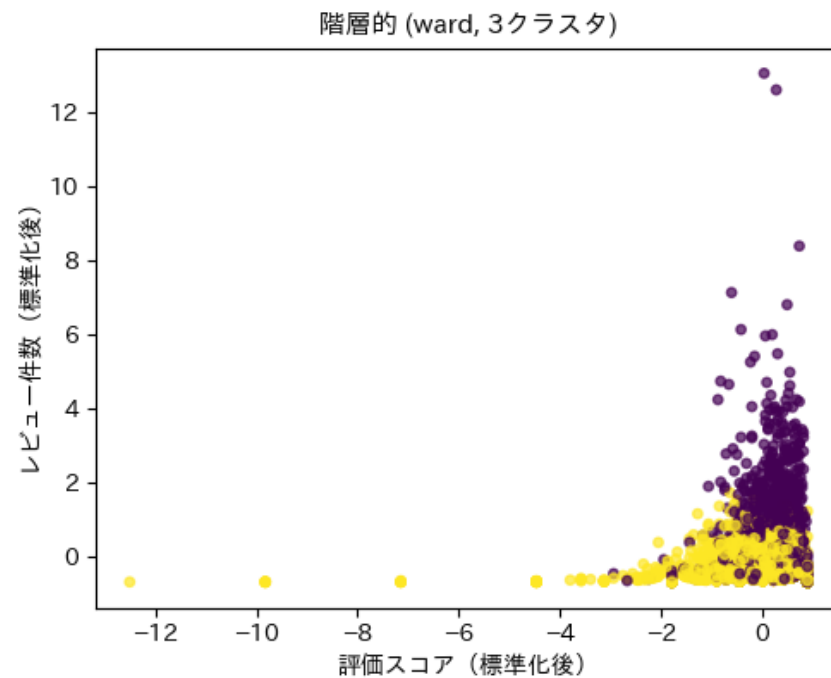
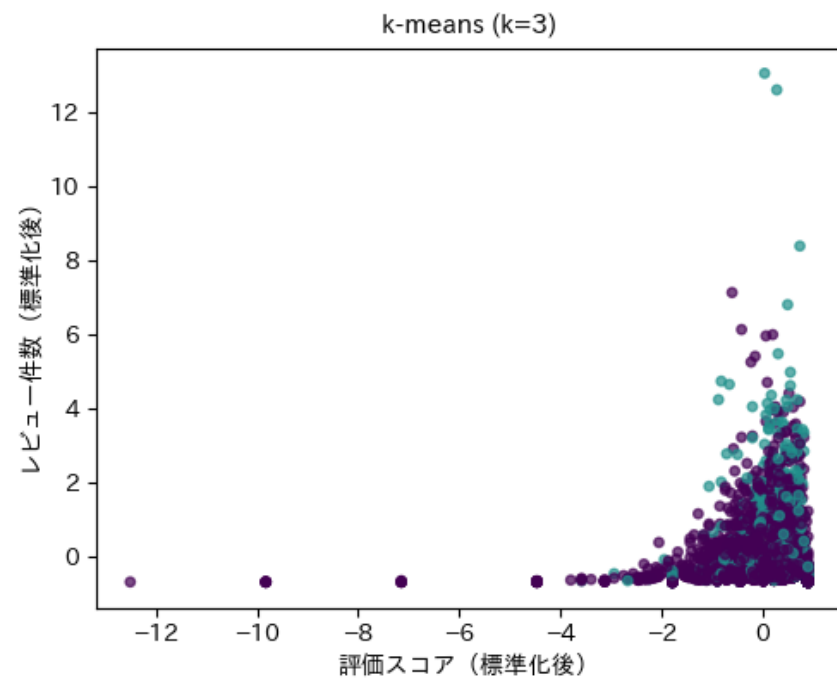
```
# 可視化用に2変数を選ぶ(レビュースコアとレビュー件数)
ix_x = FEATURES.index("review_scores_rating")
ix_y = FEATURES.index("number_of_reviews")

fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))
for ax, labels, title in [
    (axes[0], labels_km, "k-means (k=3)"),
    (axes[1], labels_hc, "階層的 (ward, 3クラスタ)"),
    (axes[2], labels_db, "DBSCAN (eps=0.5)"),
]:
    is_noise = labels == -1
    # クラスタ点(ノイズ以外)を先に・はっきり描く
    ax.scatter(X_sc[~is_noise, ix_x], X_sc[~is_noise, ix_y],
               c=labels[~is_noise], cmap="viridis", s=15, alpha=0.7)
    # ノイズは薄く・小さく、1回だけ重ねる
    if is_noise.any():
        ax.scatter(X_sc[is_noise, ix_x], X_sc[is_noise, ix_y],
                   c="red", s=9, marker="x", linewidths=0.6, alpha=0.25, label="ノイズ")
        ax.legend(fontsize=9)
    ax.set_xlabel("評価スコア(標準化後)", fontsize=10)
    ax.set_ylabel("レビュー件数(標準化後)", fontsize=10)
    ax.set_title(title, fontsize=11)

plt.suptitle("3手法のクラスタリング結果の比較", fontsize=13)
plt.tight_layout()
plt.savefig("three_methods.png", dpi=150, bbox_inches="tight")
plt.show()
```

# 3手法の比較（3）

3手法のクラスタリング結果の比較



# 前処理の重要性

## 【前処理の重要性 — クラスタリングの落とし穴】

### 標準化は必須

距離ベースの手法なので、  
変数のスケールが揃っていないと値の大きい変数だけが結果を支配する。

例：価格（数千～数万）とレビュースコア（0～5）→ 標準化しないと価格だけで決まる  
→ 解決策：StandardScalerで平均0・分散1に揃える（第7回の線形モデルと同じ）

### 次元の呪い

変数が多すぎると「すべての点が等距離に見える」現象が起き、  
クラスタリングが機能しなくなる。

高次元空間では点と点の距離の差が小さくなり、「近い・遠い」の区別が消える  
→ 解決策：次回学ぶ次元削減（PCA）で変数を圧縮してからクラスタリングする

## 【発展】教師なし vs 教師ありの照合

いろいろ試してみよう！

# k-meansのクラスタとSHラベルのクロス集計

```
cross = pd.crosstab(
    df_clean["cluster_km"],
    df_clean["is_superhost"],
    rownames=["k-meansクラスタ"],
    colnames=["スーパーホスト"]
)
print("— クラスタ × スーパーホストのクロス集計 —")
print(cross.to_string())
```

# 各クラスタのSH比率

```
print("¥n— 各クラスタのスーパーホスト比率 —")
sh_ratio = df_clean.groupby("cluster_km")["is_superhost"].mean()
for c, ratio in sh_ratio.items():
    print(f" クラスタ{c}: SH比率 {ratio:.1%}")

print(f"¥n全体のSH比率: {df_clean['is_superhost'].mean():.1%}")
print("¥n→ クラスタによってSH比率に偏りがあれば、")
print(" 教師なし学習が教師ありラベルの構造を部分的に捉えている")
```

— クラスタ × スーパーホストのクロス集計 —

| スーパーホスト     | 0    | 1   |
|-------------|------|-----|
| k-meansクラスタ |      |     |
| 0           | 1770 | 716 |
| 1           | 313  | 194 |
| 2           | 7    | 0   |

— 各クラスタのスーパーホスト比率 —

クラスタ0: SH比率 28.8%  
 クラスタ1: SH比率 38.3%  
 クラスタ2: SH比率 0.0%

全体のSH比率: 30.3%

→ クラスタによってSH比率に偏りがあれば、  
 教師なし学習が教師ありラベルの構造を部分的に捉えている



# 3

## 本日の課題

---

頑張ってください。



【課題】 期限：2026/6/17（水） 23：59まで

WebClassの『データマイニング特論』アクセスし、  
本日の日付が記載される「課題」を期限内に実施してください。

・ WebClass -> データマイニング特論 -> 【第9回（6/11）】課題

（課題1） 本日の演習で「いろいろ試してみよう！」の中で実施した入出力コードを  
html形式でエクスポートし、提出しなさい。

html形式にする手順

- ①ipynb形式でダウンロード（ファイル名の例：aaa.ipynb）
- ②再び「ファイル」へアップロード
- ③以下のコマンドを入力

```
!jupyter nbconvert --to html "aaa.ipynb"
```



## 【ADSセミナー】

本学の「先端データ科学研究センター」が主催するセミナーが6/25（木）から4週にわたって開催されます。

内容が盛りだくさんなので、**研究室を選択する前の参考になる**かと思います。

事前登録フォームから申し込みあり。

【Point】 普段の授業では聞けない話もあるかも！？

# ADSセミナー

対象 東京情報大学  
全学部・学年/大学院生/教職員

会場 1号館 201教室

【第1回】 6/25（木） 16:30 - 18:00

講師： 釣井 達也 先生  
総合情報学科 准教授（数理情報研究ユニット）



### 量子ウォークに潜む群構造について

量子ウォークはランダムウォークの量子版と解釈される数学モデルであり、その振る舞いはランダムウォークと大きく異なります。本セミナーでは、量子ウォークの簡単な具体例を通してランダムウォークとの違いを観察します。特に、量子ウォークに潜む群構造に着目し、この群構造を活用して、量子ウォークの特徴的な挙動のひとつである「周期」を求める方法について紹介します。

【第2回】 7/2（木） 16:30 - 18:00

講師： 早稲田 篤志 先生  
総合情報学科 准教授（情報セキュリティ研究ユニット）



### 個人情報保護と機械学習

プライバシーに関する情報は解析して使用することで有益な情報を得られることが期待できます。一方、漏洩等で外部に漏れると大きな問題となることがあります。そこで、基本的なプライバシー保護の指標を紹介し、機械学習との関係を考察します。

【第3回】 7/9（木） 16:30 - 18:00

講師： 西村 明 先生  
総合情報学科 教授（情報メディア学系）



### 音響法科学（オーディオ・フォレンジクス）

音響法科学は、音響学の原理と技術を、犯罪捜査・法廷および不正等の検知のために活かす研究と学問を指します。話者識別、雑音除去と音声強調、録音属性推定に大別され、本講演では、前二者を概観したのち、AI音声の判別を含む後者の研究について概説します。

【第4回】 7/16（木） 16:30 - 18:00

講師： 小早川 睦貴 先生  
総合情報学科 准教授（生命情報研究ユニット）



### 身体情報から読みとる心の働き

心理学的研究で心を読み取る際には、さまざまな人間の「反応」を測ることが不可欠です。一般的には反応時間や正答率、表情など目に見える反応を使いますが、発汗や視線など、目に見えない反応などを使って心を読む心理学的研究を紹介します。

#### <参加登録>

右のQRコードを読み取り、参加登録フォームへアクセスの上、必要事項を記入してお申し込みください。



受付期間： 5/26（火）～ 7/16（木）

<https://forms.office.com/r/JRmJe3qNmh>

早期研究体験プロジェクト  
（対象：学部1・2年次）

本プログラムに興味のある学生は、先端データ科学研究センターまでお問い合わせください。

随時  
募集中

#### <お問い合わせ>

東京情報大学 総合情報学部  
先端データ科学研究センター  
[ads@rsch.tuis.ac.jp](mailto:ads@rsch.tuis.ac.jp)

